```
    'album' => 'Slaughter of the Soul',
    'artist' => 'At the Gates',
];

knowYourClassicMetal(...$data);
```

```
Output:
Slaughter of the Soul by At the Gates, released in 1995 on Earache
```

In this example, PHP has helpful populated the function arguments that have variable names which correspond to the array keys.

Named parameters are also supported for built-in functions. The official PHP documentation will give the variable names used internally for function arguments. For example, the venerable built-in function `strpos()` has the declaration `strpos(string $haystack, string $needle, int $offset = 0): int|false`

We can call `strops` using named parameters like this:

```
print strpos(needle: 'Maiden', haystack: 'Iron Maiden Rules') . PHP_EOL;
```

```
Output:
5
```

Note that the `needle` and `haystack` arguments have had their orders reversed in the call, but because we've named them the call is still successful. Be aware, though, that there are a few rules to follow when using named parameters.

The argument names must be given as string literals, they cannot be dynamic. Which means it isn't possible to use a variable or some other expression. `someFunc($paramName: $value);` or `someFunc(getParamName(): $value)` will not work, both will result in a parse error (specifically `unexpected token ":"`) which is (happily) easily caught by a linter and should never make it to deployment. In addition, argument names can only be those found in the function signature. Any unrecognized names will generate an error.

Care must also be taken that arguments are only passed once. Duplication of named arguments isn't a syntax error, it won't be caught by a linter, and instead the code fails at run time with a fatal error. Here we duplicate the `artist` parameter: